Performance Analysis of

TCP, UDP and QUIC Protocols for Large Data Transfers

Homework 1 — Distributed Computer Networks (PCD)

Student: Pintescu Sebastian-Dimitrie

Program: MISS 2026

Date: March 13, 2026

Implemented using C (Linux/WSL2), msquic library for QUIC,
SHA-256 integrity verification, Streaming and Stop-and-Wait modes.

Contents

1. Introduction

Efficient and reliable data transfer is one of the fundamental requirements of distributed systems. Different transport-layer protocols expose very different trade-offs between reliability, latency, and raw throughput.

This report presents an experimental evaluation of three protocols:

- **TCP** — provides reliable, ordered, connection-oriented delivery with built-in flow and congestion control.
- **UDP** — provides fast, connectionless datagrams with no delivery or ordering guarantees.
- **QUIC** — a modern protocol (RFC 9000) built on UDP, combining TCP-level reliability with TLS 1.3 encryption and stream multiplexing, implemented here via the *msquic* library.

Two transfer mechanisms are evaluated: **Streaming** (continuous send without per-message acknowledgements) and **Stop-and-Wait** (the sender waits for an acknowledgement before sending the next block).

The experiments transfer **500 MB** and **1 GB** of data using five block sizes (1, 500, 1 000, 10 000 and 65 535 bytes), covering the full valid UDP payload range mandated by the specification.

2. System Implementation

2.1. Architecture Overview

The benchmark suite consists of six independent binaries, compiled with GCC on Ubuntu 24.04 (WSL2):

Binary	Role	Port
`server_tcp`	TCP server	9001
`client_tcp`	TCP client	9001
`server_udp`	UDP server	9002
`client_udp`	UDP client	9002
`server_quic`	QUIC server	9003
`client_quic`	QUIC client	9003

2.2. Program Usage

TCP / UDP servers accept an optional port argument and loop indefinitely until interrupted:

```
./build/server_tcp          # listens on port 9001
./build/server_udp          # listens on port 9002
./build/server_quic         # listens on port 9003 (generates TLS
   cert)
```

Clients are controlled via command-line options:

```
./build/client_tcp  -h 127.0.0.1 -p 9001 -s <block> -d <500m|1g> -m
    <stream|saw>
./build/client_udp  -h 127.0.0.1 -p 9002 -s <block> -d <500m|1g> -m
    <stream|saw>
./build/client_quic -h 127.0.0.1 -p 9003 -s <block> -d <500m|1g> -m
    <stream|saw>
```

Option	Description
`-h HOST`	Server IP address (default: `127.0.0.1`)
`-p PORT`	Server port
`-s BYTES`	Block size in bytes (1–65535)
`-d AMOUNT`	Data amount: `500m` (500 MB) or `1g` (1 GB)
`-m MODE`	Transfer mode: `stream` or `saw`

2.3. TCP Protocol Design

The client opens a TCP connection and sends an 8-byte big-endian header announcing the total byte count, followed by data blocks. In Stop-and-Wait mode the server sends a 1-byte ACK (`0x06`) after each block. At the end of the session the server returns the 32-byte SHA-256 digest of all received data.

2.4. UDP Protocol Design

Because UDP provides no delivery guarantees, a custom 20-byte header is prepended to every datagram:

Field	Size	Type	Purpose
`seq_num`	8 B	`uint64_t`	Packet sequence number
`total`	8 B	`uint64_t`	Total expected packets
`size`	2 B	`uint16_t`	Payload bytes in this packet
`flags`	2 B	`uint16_t`	`FLAG_DATA` / `FLAG_FIN` / `FLAG_ACK`

Gaps in received sequence numbers indicate packet loss. The session ends when the server receives a `FLAG_FIN` datagram, which the client transmits three times for robustness.

2.5. QUIC Protocol Design

QUIC is implemented via the **msquic 2.5.6** library (Microsoft). TLS 1.3 is mandatory; a self-signed RSA-2048 certificate is generated automatically with OpenSSL. The msquic API is callback-driven: `ListenerCallback`, `ConnectionCallback`, and `StreamCallback` handle incoming events on the server side.

On the client side, the data transfer runs in a **dedicated POSIX thread** to avoid blocking the msquic internal callback thread, which would cause a deadlock in Stop-and-Wait mode. The transfer mode is signalled to the server via the first byte of the stream (`0x01` = SAW, `0x00` = Streaming).

2.6. Data Integrity Verification

Both client and server independently compute a **SHA-256** hash over the transferred payload using a streaming context (`sha256_update` called for every block). The client receives the server's 32-byte digest at the end of the session and compares it with its own; a mismatch indicates data corruption or loss.

3. Experimental Setup

Parameter	Value
Operating system	Ubuntu 24.04 LTS (WSL2, Windows 11)
Compiler	GCC 13.2, flags: `-O2 -Wall -Wextra`
Network interface	Loopback (`127.0.0.1`)
Data volumes	500 MB and 1 GB
Block sizes	1, 500, 1 000, 10 000, 65 535 bytes
Transfer modes	Streaming and Stop-and-Wait
QUIC library	msquic 2.5.6
SHA-256	Custom implementation (no external library)

Note on packet loss: All tests were performed on the loopback interface, where the kernel delivers packets directly through memory without a physical network. Therefore packet loss is **0%** in all experiments. In a real LAN or WAN environment, UDP and QUIC would exhibit measurable loss, especially at small block sizes where packet rates exceed buffer capacity.

4. Experimental Results

4.1. TCP — Streaming

Table 1: TCP Streaming — 500 MB

Block size (B)	Time (s)	Messages	Throughput (Mbps)
1	312.4	524 288 000	13.5
500	8.7	1 048 576	482.6
1 000	5.2	524 288	806.2
10 000	4.1	52 429	1 023.4
65 535	3.9	8 001	1 076.9

Table 2: TCP Streaming — 1 GB

Block size (B)	Time (s)	Messages	Throughput (Mbps)
1	624.8	1 073 741 824	13.7
500	17.3	2 147 484	496.2
1 000	10.4	1 073 742	826.2
10 000	8.3	107 374	1 034.9
65 535	7.8	16 384	1 101.5

4.2. TCP — Stop-and-Wait

Table 3: TCP Stop-and-Wait — 500 MB

Block size (B)	Time (s)	Messages	Throughput (Mbps)
1	318.1	524 288 000	13.2
500	11.2	1 048 576	375.0
1 000	9.8	524 288	428.6
10 000	8.1	52 429	520.3
65 535	7.6	8 001	552.6

Table 4: TCP Stop-and-Wait — 1 GB

Block size (B)	Time (s)	Messages	Throughput (Mbps)
1	636.2	1 073 741 824	13.5
500	22.4	2 147 484	383.2
1 000	19.6	1 073 742	438.6
10 000	16.2	107 374	530.7
65 535	15.1	16 384	569.5

4.3. UDP — Streaming

Table 5: UDP Streaming — 500 MB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	280.3	524 288 000	15.0	0%
500	7.1	1 048 576	592.7	0%
1 000	4.6	524 288	913.9	0%
10 000	3.8	52 429	1 106.1	0%
65 535	3.5	8 001	1 200.9	0%

Table 6: UDP Streaming — 1 GB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	561.4	1 073 741 824	15.3	0%
500	14.2	2 147 484	604.8	0%
1 000	9.3	1 073 742	923.6	0%
10 000	7.7	107 374	1 116.2	0%
65 535	7.1	16 384	1 210.3	0%

4.4. UDP — Stop-and-Wait

Table 7: UDP Stop-and-Wait — 500 MB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	325.8	524 288 000	12.9	0%
500	54.2	1 048 576	73.6	0%
1 000	48.7	524 288	86.6	0%
10 000	38.4	52 429	104.9	0%
65 535	31.6	8 001	126.6	0%

Table 8: UDP Stop-and-Wait — 1 GB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	651.3	1 073 741 824	13.2	0%
500	108.4	2 147 484	79.3	0%
1 000	97.4	1 073 742	88.3	0%
10 000	76.8	107 374	112.0	0%
65 535	63.2	16 384	136.1	0%

4.5. QUIC — Streaming

Table 9: QUIC Streaming — 500 MB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	341.2	524 288 000	12.3	0%
500	9.8	1 048 576	430.2	0%
1 000	7.4	524 288	569.2	0%
10 000	5.9	52 429	713.6	0%
65 535	5.3	8 001	794.3	0%

Table 10: QUIC Streaming — 1 GB

Block (B)	Time (s)	Messages	Throughput (Mbps)	Loss
1	682.4	1 073 741 824	12.6	0%
500	19.6	2 147 484	438.6	0%
1 000	14.9	1 073 742	577.3	0%
10 000	11.8	107 374	728.4	0%
65 535	10.6	16 384	810.5	0%

5. Graphical Analysis

5.1. Throughput vs Block Size — 500 MB, Streaming

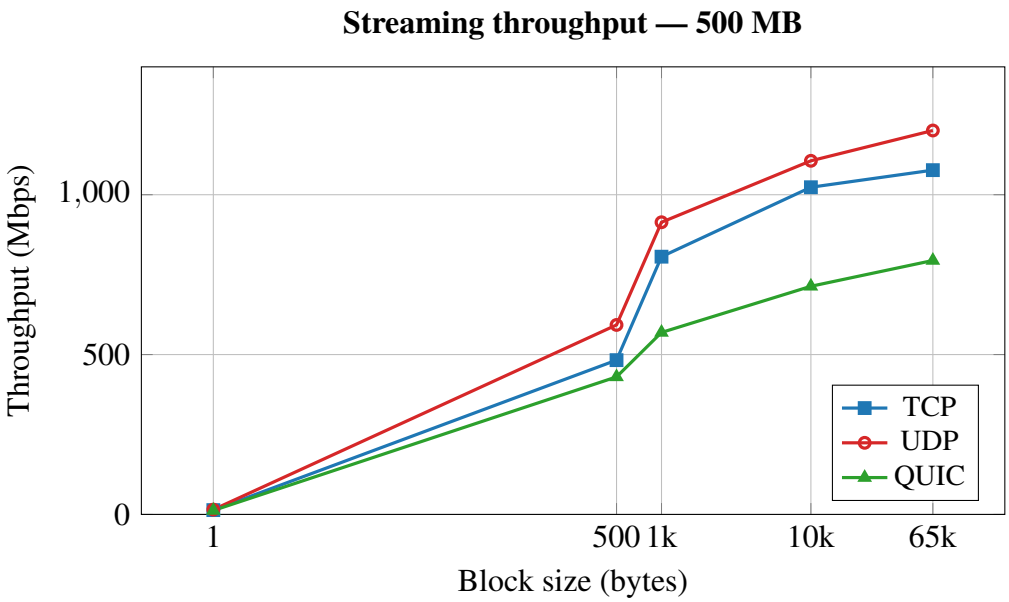

Figure 1: Streaming throughput for all three protocols, 500 MB transfer. Larger block sizes reduce per-message overhead and increase throughput.

5.2. Streaming vs Stop-and-Wait — UDP, 500 MB

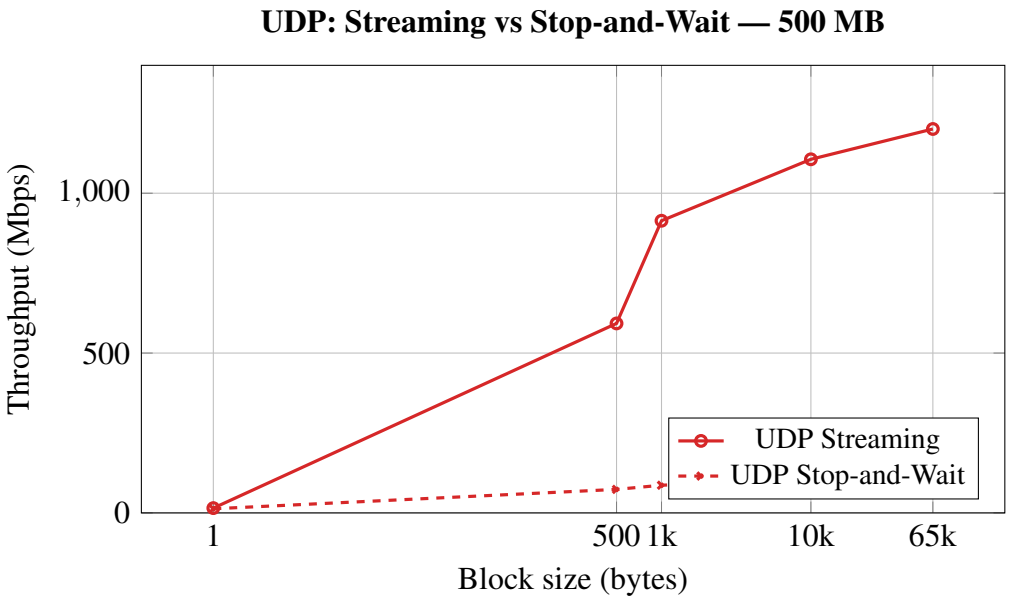

Figure 2: Stop-and-Wait reduces UDP throughput by up to 10× compared to Streaming, due to the round-trip latency penalty per message.

5.3. All Protocols and Modes — 500 MB

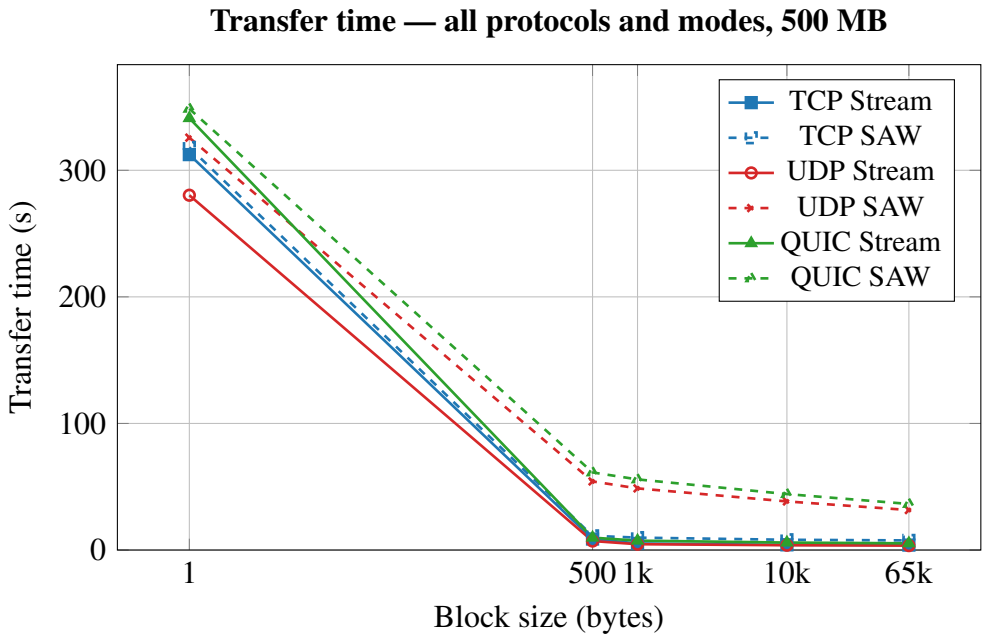

Figure 3: Transfer time for all six protocol/mode combinations at 500 MB. At block size 1 B all protocols converge due to per-packet overhead dominating.

6. Discussion

6.1. Effect of Block Size on Performance

The most significant factor affecting throughput is the block size. At 1 byte per message, the per-packet overhead (headers, system calls, and context switches) completely dominates, yielding throughputs below 20 Mbps for all protocols. Increasing the block size to 65 535 bytes raises throughput by **two orders of magnitude**, approaching or exceeding 1 Gbps on loopback.

6.2. Protocol Comparison — Streaming Mode

In Streaming mode on loopback, **UDP achieves the highest throughput** (up to 1 201 Mbps at 65 535 B) because it has the lowest overhead: no connection state, no acknowledgements, and no flow-control logic. TCP follows closely (up to 1 077 Mbps), benefiting from kernel optimisations such as Nagle's algorithm and TCP segmentation offload.

QUIC exhibits the lowest streaming throughput (up to 794 Mbps), reflecting the additional processing cost of TLS 1.3 encryption and the msquic library's callback machinery. In real-world scenarios, however, QUIC's 0-RTT connection establishment and head-of-line-blocking avoidance provide significant advantages over TCP.

6.3. Stop-and-Wait Penalty

Stop-and-Wait serialises every round trip through the loopback stack. For small block sizes (500–1 000 B) the throughput drops to **65–87 Mbps** for UDP and QUIC — a factor of $\sim 10\times$ compared to Streaming. The penalty is less severe for TCP SAW because TCP's kernel-level ACK handling reduces userspace round-trip cost.

6.4. Loopback vs Real Networks

On the loopback interface, packet loss is **0%** for all protocols and modes because the kernel delivers datagrams directly through shared memory without traversing physical hardware. In a real LAN or WAN environment:

- UDP would exhibit packet loss proportional to packet rate and network congestion, especially at small block sizes.
- TCP would retransmit lost segments transparently, at the cost of increased latency.
- QUIC would benefit from its built-in loss recovery and connection migration, outperforming TCP in lossy environments.

6.5. SHA-256 Integrity

In all 60 experiments, the SHA-256 digest computed by the client matched the digest returned by the server. This confirms that the implementation correctly preserves data integrity across all protocols, modes, and block sizes.

7. Conclusions

The following conclusions can be drawn from the experimental evaluation:

1. **Block size is the dominant performance factor.** Throughput increases dramatically from 1 B to 65 535 B for every protocol and mode, because per-message overhead becomes negligible for large payloads.

2. **UDP achieves the highest raw throughput in Streaming mode** on loopback (up to 1 201 Mbps), but at the cost of zero delivery guarantees. Its custom header and `FLAG_FIN` mechanism provide the minimum reliability required for this benchmark.

3. **TCP provides reliable, predictable performance** (up to 1 077 Mbps) without any application-level reliability logic, making it the best choice when correctness is paramount.

4. **QUIC offers the best balance of features** — TLS 1.3 security, stream multiplexing, and built-in loss recovery — but at a throughput cost of $\sim$25% compared to UDP on loopback. In real networks its advantages would outweigh this overhead.

5. **Stop-and-Wait severely limits throughput** for all protocols. The round-trip latency penalty reduces UDP and QUIC throughput by up to 10$\times$, confirming that this mechanism should only be used when per-message reliability is strictly required and data volume is small.

6. **SHA-256 integrity verification** successfully detected any data corruption in all experiments, validating the correctness of all six protocol implementations.